

Esercitazione: Navigazione e Manipolazione del File System

Preparazione dell'ambiente

Prima di scrivere il codice, prepara il progetto in PyCharm:

1. Crea una cartella principale chiamata `EsercizioPF02`.
2. Dentro `EsercizioPF02`, crea due sottocartelle: `codice` e `dati`.
3. Crea un file Python chiamato `gestore_file.py` dentro la cartella `codice`.

La struttura finale deve essere questa:

```
EsercizioPF02/  
├── codice/  
│   └── gestore_file.py    ← lo script che scriveremo  
└── dati/                 ← qui verranno creati i file
```

1. Un problema: dove "si trova" Python quando esegue lo script?

Partiamo dal codice più intuitivo possibile. Scrivi in `gestore_file.py`:

```
import os  
  
percorso_attuale = os.getcwd()  
print(f"Mi trovo in: {percorso_attuale}")
```

Esegui lo script da PyCharm e osserva l'output. Poi fai questo esperimento:

1. Apri il **Terminale** di PyCharm (in basso).
2. Spostati dentro la cartella `codice` con `cd codice`.
3. Lancia lo script con `python gestore_file.py`.

Cosa noti? L'output è diverso a seconda di come lanci lo script.

=> `os.getcwd()` non dice "dove si trova lo script". Dice **da quale cartella è stato lanciato il processo Python**. Si chiama *Current Working Directory* (CWD), cartella di lavoro corrente, ed

è uno stato del processo, non dello script.

Questo è un problema: se lo script ha bisogno di leggere o scrivere file nella cartella `dati/`, non possiamo basarci su `os.getcwd()`. Ci serve un modo per trovare la cartella `dati/` **rispetto alla posizione dello script**, non rispetto a chi lo lancia.

2. La soluzione: la variabile speciale `__file__`

Ogni modulo Python ha una variabile speciale chiamata `__file__` che contiene il percorso del file `.py` stesso. Da lì possiamo ricostruire qualsiasi posizione nel progetto.

```
import os

# Percorso assoluto del file gestore_file.py
percorso_script = os.path.abspath(__file__)
print(f"Lo script è: {percorso_script}")

# Cartella che contiene lo script: codice/
cartella_script = os.path.dirname(percorso_script)
print(f"La cartella dello script è: {cartella_script}")

# Un livello sopra: la radice del progetto EsercizioPF02/
radice_progetto = os.path.dirname(cartella_script)
print(f"La radice del progetto è: {radice_progetto}")
```

Output atteso (su Windows):

```
Lo script è:
C:\Users\studente\PycharmProjects\EsercizioPF02\codice\gestore_file.py
La cartella dello script è:
C:\Users\studente\PycharmProjects\EsercizioPF02\codice
La radice del progetto è: C:\Users\studente\PycharmProjects\EsercizioPF02
```

=> **Rilancia lo script da terminale dopo aver fatto `cd` in cartelle diverse: l'output non cambia. `__file__` è ancorato al file, non al processo.**

Riepilogo delle funzioni usate finora

Funzione	Cosa fa
<code>os.path.abspath(path)</code>	Trasforma un percorso (anche relativo) in percorso assoluto
<code>os.path.dirname(path)</code>	Restituisce la cartella che contiene il file/cartella indicato
<code>__file__</code>	Variabile speciale: percorso del file <code>.py</code> corrente

3. Costruire il percorso verso la cartella dati

Ora che abbiamo la radice del progetto, il percorso verso `dati/` si ottiene unendo la radice con il nome della cartella. Per farlo **non usiamo la concatenazione di stringhe**, ma la funzione `os.path.join()`:

```
cartella_dati = os.path.join(radice_progetto, "dati")
print(f"Percorso della cartella dati: {cartella_dati}")
```

=> **Perché `os.path.join()` e non `radice_progetto + "/dati"`?** Perché il separatore dei percorsi è diverso tra sistemi operativi: `/` su Linux e Mac, `\` su Windows. Se scrivessi `"dati/anni_input.txt"` a mano, il codice funzionerebbe oggi su Linux ma potrebbe rompersi su Windows con percorsi più complessi. `os.path.join()` sceglie automaticamente il separatore corretto per il sistema su cui sta girando lo script.

Prima di usare la cartella, verifichiamo che esista davvero e che sia una directory:

```
if os.path.isdir(cartella_dati):
    print("Cartella 'dati' trovata!")
else:
    print("Errore: la cartella 'dati' non esiste. Creala a mano!")
    exit(1)
```

💡 **Perché `os.path.isdir()` e non `"dati" in os.listdir(radice_progetto)`?**

`os.listdir()` restituisce solo i nomi, senza distinguere file da cartelle. Se per sbaglio esistesse un *file* chiamato `dati` nella radice, la condizione `in` sarebbe `True` lo stesso e lo script proverebbe a scriverci dentro fallendo. `isdir()` verifica che sia proprio una directory.

4. Esplorare il contenuto del progetto

Prima di creare i file, diamo un'occhiata a cosa c'è nelle cartelle. Usiamo `os.listdir()` per ottenere la lista dei nomi contenuti:

```
print("\nContenuto della radice del progetto:")
print(os.listdir(radice_progetto))
```

```
print("\nContenuto della cartella dati:")
print(os.listdir(cartella_dati))
```

Output atteso (prima di creare i file):

```
Contenuto della radice del progetto:
['codice', 'dati']
```

```
Contenuto della cartella dati:
[]
```

=> `os.listdir()` restituisce una **lista di stringhe** con i nomi (non i percorsi completi) di file e cartelle. La cartella `dati` è inizialmente vuota.

5. Creare i file di testo dentro `dati`

Per ogni file, costruiamo il percorso completo con `os.path.join()`, unendo il percorso di `dati` e il nome del file:

```
file_da_generare = ["test_1.txt", "test_2.txt", "log_sistema.txt"]

for nome in file_da_generare:
    percorso_file = os.path.join(cartella_dati, nome)
    with open(percorso_file, "w", encoding="utf-8") as f:
        f.write(f"File generato automaticamente: {nome}\n")
        f.write("Pronto per fare tante cose belle.")

print(f"\nOperazione completata. Creati {len(file_da_generare)} file.")
```

Nota che `open()` accetta il percorso completo: non importa da dove è stato lanciato lo script, il file verrà creato sempre nel posto giusto.

Codice completo

```
import os

# 1. Troviamo la posizione dello script e la radice del progetto
percorso_script = os.path.abspath(__file__)
cartella_script = os.path.dirname(percorso_script)
```

```

radice_progetto = os.path.dirname(cartella_script)

print(f"Radice del progetto: {radice_progetto}")

# 2. Costruiamo il percorso verso 'dati' e verifichiamo che esista
cartella_dati = os.path.join(radice_progetto, "dati")

if not os.path.isdir(cartella_dati):
    print("Errore: la cartella 'dati' non esiste. Creala a mano!")
    exit(1)

print(f"Cartella dati trovata: {cartella_dati}")

# 3. Esploriamo il contenuto prima di scrivere
print("\nContenuto iniziale della cartella dati:")
print(os.listdir(cartella_dati))

# 4. Creiamo i file
file_da_generare = ["test_1.txt", "test_2.txt", "log_sistema.txt"]

for nome in file_da_generare:
    percorso_file = os.path.join(cartella_dati, nome)
    with open(percorso_file, "w", encoding="utf-8") as f:
        f.write(f"File generato automaticamente: {nome}\n")
        f.write("Pronto per fare tante cose belle.")

print(f"\nOperazione completata. Creati {len(file_da_generare)} file.")
print("\nContenuto finale della cartella dati:")
print(os.listdir(cartella_dati))

```

Riepilogo delle funzioni usate

Funzione	Cosa fa
<code>__file__</code>	Variabile speciale: percorso del file <code>.py</code> corrente
<code>os.path.abspath(path)</code>	Trasforma un percorso in percorso assoluto
<code>os.path.dirname(path)</code>	Restituisce la cartella contenitore di <code>path</code>
<code>os.path.join(a, b)</code>	Costruisce un percorso unendo <code>a</code> e <code>b</code> con il separatore corretto del sistema operativo
<code>os.path.isdir(path)</code>	Restituisce <code>True</code> se <code>path</code> è una directory esistente
<code>os.listdir(path)</code>	Restituisce la lista dei nomi contenuti in <code>path</code>

Funzione	Cosa fa
<code>os.getcwd()</code>	Restituisce la CWD (cartella di lavoro corrente del processo) — da evitare per individuare file relativi allo script

Esercizi

1. **Esplora** — modifica il codice per stampare separatamente file e cartelle contenuti nella radice del progetto. Suggerimento: scorri `os.listdir(radice_progetto)` con un `for` e per ogni nome costruisci il percorso completo con `os.path.join()`, poi usa `os.path.isfile()` e `os.path.isdir()` per distinguerli.
 1. se il percorso che incontri rappresenta un file, prova a leggerlo.
2. **Verifica** — dopo aver creato i file, usa `os.listdir(cartella_dati)` per confermare che i tre file siano effettivamente presenti. Conta quanti sono e confronta con `len(file_da_generare)`.
3. **Sperimenta con la CWD** — aggiungi in testa allo script una riga che stampi `os.getcwd()`. Poi lancia lo script in tre modi diversi:
 - da PyCharm con il pulsante Run
 - da terminale, posizionato nella radice del progetto: `python codice/gestore_file.py`
 - da terminale, posizionato dentro `codice/`: `python gestore_file.py`

Osserva che `os.getcwd()` cambia, ma il percorso costruito con `__file__` **no**. Spiega a parole perché questo rende il nostro script più affidabile.

4. **Portabilità** — perché usiamo `os.path.join()` invece di scrivere il percorso direttamente come stringa (es. `"dati/test_1.txt"`)? Cosa succederebbe, in linea di principio, se questo script venisse eseguito su Windows e poi copiato su un Mac? E se un percorso contenesse spazi o caratteri speciali?